Reporte

Nombre: Roberto Carlos Sánchez Flores Donovan Adrian Serrato Lores Laura Sofía Vargas Ramírez	**Matrícula**: 2151262 2154304 2152053
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Laboratorio guiado: programa para administrar pedidos
Fecha: 28/10/2026	
Bibliografia (Formato APA): Documentos dados por el docente: Manual de lectura ilustrado (Unidad 4), Práctica guiada TechStore y paquete CLN_U4_Paquete_Alumno.zip. Evans, E. (2003). Domain-Driven Design: Tackling complexity in the heart of software. Addison-Wesley. Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley. Python Software Foundation. (2026). decimal — Decimal fixed-point and floating-point arithmetic. https://docs.python.org/3/library/decimal.html Ramírez, S. (2026). FastAPI documentation. https://fastapi.tiangolo.com/ pytest-dev. (2026). pytest documentation. https://docs.pytest.org/	

Objetivo:

Implementar el núcleo de negocio de una tienda ficticia, TechStore, a partir de los datos del paquete del alumno. El programa lee clientes, productos y pedidos de archivos CSV y los convierte en objetos del dominio; protege las invariantes con entidades y Value Objects; evalúa las reglas de descuento con el Specification Pattern y su composición AND, OR y NOT; controla el ciclo de vida de cada pedido con una máquina de estados que registra Domain Events, y expone las operaciones mediante una API REST con FastAPI. Las reglas se demuestran ejecutando los 15 escenarios del paquete (E01–E15) y con pruebas automáticas en pytest.

Procedimiento:

TechStore — Núcleo de negocio de pedidos

En la Unidad 3 se diseñaron responsabilidades y se identificaron reglas de negocio; en esta unidad ese diseño se convierte en clases, métodos, estados y pruebas. La pregunta principal deja de ser qué objetos existen y pasa a ser qué operaciones son válidas y quién tiene autoridad para ejecutarlas. Por eso el pedido no se modifica asignando un estado desde afuera: se le pide `order.confirm()`, y es el propio pedido el que decide si la operación procede.

1. Material recibido

Archivo	Contenido	Uso en el proyecto
datos/clientes.csv	12 clientes; es_vip y activo (1 = sí, 0 = no)	Objetos Customer
datos/productos.csv	12 productos con precio_mxn y existencia	Precio de cada OrderItem (Decimal)
datos/pedidos.csv	16 pedidos, todos en DRAFT, con tiene_promocion	Objetos Order
datos/detalle_pedidos.csv	21 renglones (pedido, SKU, cantidad)	OrderItem agregados con add_item()
datos/escenarios.csv	15 casos E01–E15 con acción y tipo de resultado	Bitácora y pruebas de escenarios
cargar_csv.py	Cargador que importa las clases del dominio	Se usó sin modificaciones

Tabla 1. *Archivos del paquete CLN_U4_Paquete_Alumno.zip.*

La existencia de productos.csv es solo información del catálogo: la práctica no exige descontar inventario. Los datos se mantienen en memoria, por lo que los cambios se pierden al cerrar el programa; no se usó base de datos.

2. Preparación del ambiente

Se instaló Python 3.13.17 desde python.org con la opción «Add python.exe to PATH», se creó un entorno virtual y se instalaron las dependencias. Se eligió la versión 3.13 —la misma con la que se generó el paquete del docente— en lugar de la más reciente, para evitar incompatibilidades de las librerías con una versión recién publicada.

Terminal (PowerShell)

```
python --version                      # Python 3.13.17
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt   # fastapi, uvicorn, pytest, httpx
```

Estructura final del proyecto:

```
techstore-u4/
├── datos/                ← 5 CSV del paquete (sin modificar)
├── cargar_csv.py         ← cargador del paquete (sin modificar)
├── ejecutar_escenarios.py ← ejecuta y compara E01–E15
├── app/
│   ├── domain.py         ← entidades, Value Objects, estados, eventos
│   ├── specifications.py ← Specifications y reglas de descuento
│   ├── services.py       ← Application Service y repositorio
│   └── api.py            ← endpoints FastAPI y códigos HTTP
├── tests/                ← 4 archivos de pruebas pytest
├── docs/                 ← diagrama de estados (.drawio y .png)
└── README.md, requirements.txt, pytest.ini, .gitignore
```

Las dependencias apuntan hacia el dominio: `api → services → domain`. El dominio no conoce FastAPI ni los CSV, por lo que puede ejecutarse y probarse sin servidor web.

3. Lectura de los CSV y cálculo manual

cliente_id de pedidos.csv apunta a clientes.csv; en detalle_pedidos.csv, pedido_id enlaza con el pedido y sku con productos.csv. El subtotal de un renglón es cantidad × precio y el total del pedido es la suma de sus subtotales. Antes de programar se calcularon a mano tres pedidos:

Pedido	Cliente	Renglones	Unidades	Total manual
O001	C001 · Ana Torres (VIP, activa)	SKU001 × 2 × 600.00 = 1,200.00 SKU002 × 2 × 450.00 = 900.00	4	2,100.00 MXN
O006	C006 · Pedro Soto (no VIP)	SKU008 × 1 × 18,000.00 = 18,000.00	1	18,000.00 MXN
O010	C010 · Jorge Luna (no VIP)	SKU003 × 1 × 120.00 = 120.00 SKU009 × 1 × 380.00 = 380.00	2	500.00 MXN

Tabla 2. *Cálculo manual de la sección 5 de la práctica.*

Los tres resultados coinciden con lo que después mostró el programa y quedaron como prueba automática (test_calculo_manual_de_la_seccion_5). O010 ilustra la diferencia entre contar renglones y sumar unidades: dos renglones de una unidad cada uno; O011 tiene dos renglones pero tres unidades.

4. Dominio: entidades, Value Objects e invariantes

El archivo app/domain.py concentra las reglas. Cada clase garantiza en su constructor que el objeto siempre es válido, y los métodos de Order expresan lo que el negocio puede hacer.

Clase	Tipo	Invariantes que protege
Money	Value Object inmutable (frozen)	amount es Decimal (rechaza float), no negativo y finito; add() solo suma la misma moneda y devuelve un objeto nuevo.
Customer	Value Object inmutable	Identificador obligatorio; is_vip y active vienen de clientes.csv.
OrderItem	Value Object inmutable	SKU no vacío, cantidad entera mayor que cero; el subtotal se calcula, no se captura.
Order	Entity / Aggregate Root	Máximo 20 unidades (suma de cantidades); solo DRAFT admite artículos; transiciones de estado controladas; eventos solo tras una transición válida.
OrderStatus	Enumeración	DRAFT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED y RETURNED.
DomainEvent	Objeto inmutable	name, order_id, customer_id, total, detail (guía o motivo) y occurred_at UTC.
Excepciones	Jerarquía	DomainException → OrderException → InvalidOrderData, InvalidOrderTransition, OrderItemLimitExceeded.

Tabla 3. *Clases del dominio.*

app/domain.py — Money

```
@dataclass(frozen=True)
class Money:
    amount: Decimal
    currency: str = "MXN"


    def __post_init__(self):
        if not isinstance(self.amount, Decimal):
            raise InvalidOrderData("El monto debe ser Decimal")
        if not self.amount.is_finite() or self.amount < 0:
            raise InvalidOrderData("Monto inválido")


    def add(self, other):
        if self.currency != other.currency:
            raise InvalidOrderData("No se pueden sumar monedas distintas")
        return Money(self.amount + other.amount, self.currency)
```

app/domain.py — Order.add_item

```
def add_item(self, item):
    if self.status != OrderStatus.DRAFT:
        raise InvalidOrderTransition("agregar artículos a", self.status)
    ...
    if self.units + item.quantity > MAX_UNITS_PER_ORDER:   # unidades
        raise OrderItemLimitExceeded("Máximo 20 unidades ...")
    self.items.append(item)            # solo si todo lo anterior pasó
```

Si un pedido tiene 19 unidades y se intenta agregar 2, add_item() se detiene antes de modificar la lista de artículos, de modo que el pedido queda exactamente igual.

5. Primer punto de control: carga de los 16 pedidos

Con las clases del dominio creadas, el cargador del paquete funcionó sin cambios. Muestra una línea por pedido con su identificador, estado, unidades y total; los 16 aparecen en DRAFT y O001 muestra 4 unidades y 2100.00 MXN.

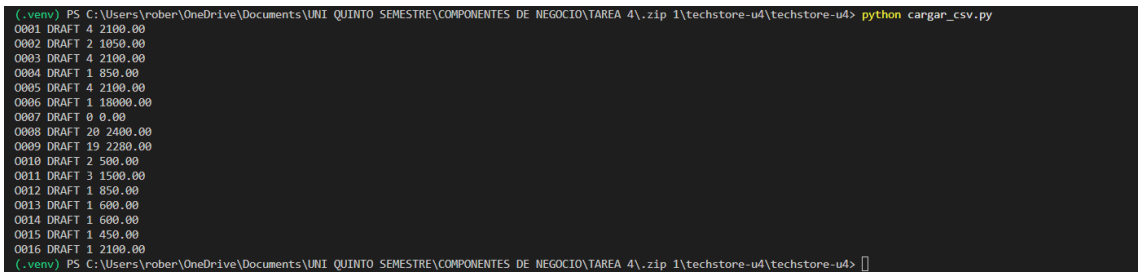


Figura 1. *Ejecución de python cargar_csv.py: 16 pedidos cargados en DRAFT.*

6. Specification Pattern y reglas de descuento

En app/specifications.py cada regla se convirtió en un objeto con el método is_satisfied_by(order), que responde verdadero o falso sin modificar el pedido. Los operadores & (AND), | (OR) y ~ (NOT) crean AndSpec, OrSpec y NotSpec, de modo que las reglas se combinan sin repetir if anidados.

Specification	Condición	Detalle importante
MinimumAmountSpec(n)	total > n	Estricto: usa > y no >=. Un total de exactamente 500 no supera el umbral.
VIPCustomerSpec	cliente VIP	Lee customer.is_vip.
MinimumUnitsSpec(n)	unidades ≥ n	Suma cantidades, no cuenta renglones.
ActivePromotionSpec	promoción activa	Se usa negada (~) en ambas reglas.

Tabla 4. *Specifications implementadas.*

app/specifications.py — reglas compuestas

```
TEN_PERCENT_RULE = (MinimumAmountSpec(Decimal("500")) & VIPCustomerSpec()
                    & MinimumUnitsSpec(4) & ~ActivePromotionSpec())
FIVE_PERCENT_RULE = MinimumAmountSpec(Decimal("5000")) & ~ActivePromotionSpec()
ANY_DISCOUNT_RULE = TEN_PERCENT_RULE | FIVE_PERCENT_RULE


def evaluate_discount(order):
    if not ANY_DISCOUNT_RULE.is_satisfied_by(order):
        percent = 0
    elif TEN_PERCENT_RULE.is_satisfied_by(order):   # prioridad 10 %
        percent = 10
    else:
        percent = 5
```

Pedido	Datos	10 %	5 %	Resultado
O001	VIP, 2,100.00, 4 u, sin promoción	Sí	No	10 % → paga 1,890.00

Pedido	Datos	10 %	5 %	Resultado
O002	No VIP, 1,050.00, 2 u	No	No	0 %
O005	VIP, 2,100.00, 4 u, con promoción	No (NOT promo)	No	0 %
O006	No VIP, 18,000.00, 1 u, sin promoción	No	Sí	5 % → paga 17,100.00
O010	No VIP, exactamente 500.00, 2 u	No (500 no es > 500)	No	0 %
O011	VIP, 1,500.00, 3 u	No (requiere 4)	No	0 %

Tabla 5. *Evaluación de los descuentos de los escenarios E01, E02, E04, E05, E09 y E10.*

Para comprobar la prioridad se agregó una prueba con un pedido VIP de 8,000 MXN y 4 unidades: cumple ambas reglas y recibe 10 %.

7. Máquina de estados y Domain Events

Order tiene los métodos confirm(), ship(tracking_number), cancel(reason), deliver() y return_order(). Todos siguen el mismo orden: validar primero, cambiar el estado después y registrar el evento al final. Si una validación falla se lanza una excepción y el pedido conserva su estado y su lista de eventos.

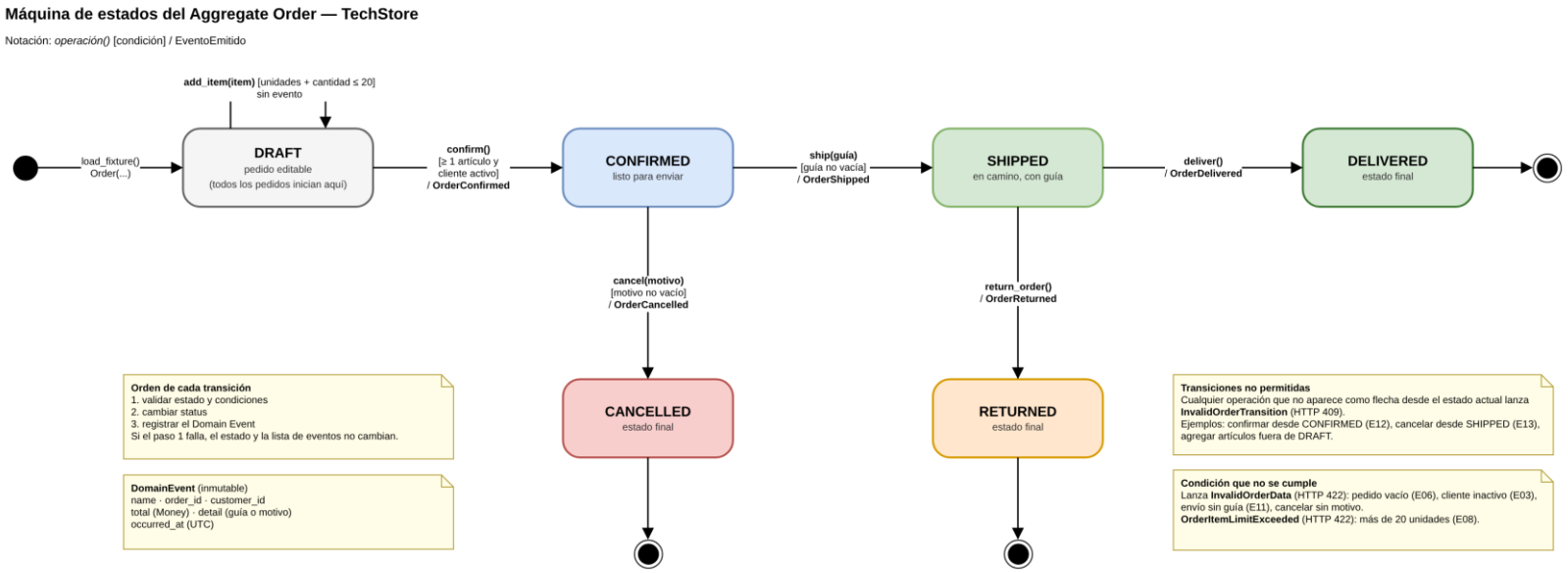


Figura 2. *Diagrama de estados del Aggregate Order (docs/diagrama-estados.drawio).*

Operación	Desde	Hacia	Condición	Si no se cumple	Evento
confirm()	DRAFT	CONFIRMED	Al menos un artículo y cliente activo	InvalidOrderData (422)	OrderConfirmed
ship(guía)	CONFIRMED	SHIPPED	Número de guía no vacío	InvalidOrderData (422)	OrderShipped
cancel(motivo)	CONFIRMED	CANCELLED	Motivo no vacío	InvalidOrderData (422)	OrderCancelled
deliver()	SHIPPED	DELIVERED	—	—	OrderDelivered
return_order()	SHIPPED	RETURNED	—	—	OrderReturned
Cualquier otra	—	—	La transición no existe en el diagrama	InvalidOrderTransition (409)	Ninguno

Tabla 6. *Transiciones permitidas, condiciones y eventos.*

Explicación de los Domain Events. Un Domain Event comunica que un hecho del negocio ya ocurrió; por eso su nombre está en pasado (OrderConfirmed, OrderShipped, OrderCancelled, OrderDelivered, OrderReturned). Cada evento es un objeto inmutable con el identificador del pedido y del cliente, el total, un detalle (la guía en el envío y el motivo en la cancelación) y la fecha y hora UTC. El pedido los acumula en su lista events; en un sistema real, el repositorio o la unidad de trabajo los publicaría al guardar para que otros procesos reaccionen —enviar un correo, iniciar la facturación o actualizar inventario— sin que Order conozca esas implementaciones. En este laboratorio no se integran correo ni inventario: los eventos quedan registrados y se consultan en las pruebas y en la respuesta de la API.

app/domain.py — confirmación

```
def confirm(self):
    self._require_status("confirmar", OrderStatus.DRAFT)   # 1. validar
    if not self.items:
        raise InvalidOrderData("... pedido sin artículos")
    if not self.customer.active:
        raise InvalidOrderData("... el cliente está inactivo")
    self._change_to(OrderStatus.CONFIRMED, "OrderConfirmed")   # 2 y 3


def _change_to(self, new_status, event_name, detail=""):
    self.status = new_status
    self.events.append(DomainEvent(event_name, self.id, self.customer.id,
                                   self.total, detail))
```

8. Ejecución de los escenarios E01–E15

Cada escenario se ejecutó con objetos cargados desde load_fixture(), usando una carga nueva para cada caso de modo que los cambios de estado de uno no afecten a otro. No se modificaron los CSV ni las reglas. El script ejecutar_escenarios.py corre los 15 casos y compara el resultado con la columna objetivo de escenarios.csv (15/15 coinciden). Cada caso tiene su prueba en tests/test_escenarios.py; el nombre de la prueba aparece en la primera columna de la bitácora.

Bitácora de escenarios E01–E15

Caso y prueba	Precondiciones	Operación	Resultado esperado	Resultado real	Evento	Explicación
E01 **O001** **test_e01**	DRAFT. Cliente C001 VIP y activo; 4 u; 2,100.00; sin promoción	Evaluar descuento	10 %	10 %: descuento 210.00, paga 1,890.00. Estado sin cambios.	Ninguno (consulta)	Cumple las cuatro condiciones del AND.
E02 **O002** **test_e02**	DRAFT. C002 no VIP; 2 u; 1,050.00; sin promoción	Evaluar descuento	0 %	0 %: paga 1,050.00.	Ninguno (consulta)	No es VIP y el total no supera 5,000.
E03 **O003** **test_e03**	DRAFT. C003 VIP pero inactivo; 4 u; 2,100.00	`confirm()`	Rechazo	InvalidOrderData: el cliente C003 está inactivo. Sigue en DRAFT.	Sin evento	Solo un cliente activo puede confirmar.
E04 **O005** **test_e04**	DRAFT. C005 VIP; 4 u; 2,100.00; con promoción	Evaluar descuento	0 %	0 %: paga 2,100.00.	Ninguno (consulta)	La negación de la promoción no se cumple: la promoción excluye descuentos.
E05 **O006** **test_e05**	DRAFT. C006 no VIP; 1 u; 18,000.00; sin promoción	Evaluar descuento	5 %	5 %: descuento 900.00, paga 17,100.00.	Ninguno (consulta)	Falla la regla del 10 % pero se cumple la del 5 % (OR).

Caso y prueba	Precondiciones	Operación	Resultado esperado	Resultado real	Evento	Explicación
E06 O007 test_e06	DRAFT. C007 activo; 0 artículos; 0.00	`confirm()`	Rechazo	InvalidOrderData: pedido sin artículos. Sigue en DRAFT.	Sin evento	Un pedido vacío no puede confirmarse.
E07 O008 test_e07	DRAFT. C008 activo; 20 u (SKU003 × 20); 2,400.00	`confirm()`	Éxito	Pasa a CONFIRMED.	OrderConfirmed	El límite es 20 inclusive: 20 unidades están permitidas.
E08 O009 test_e08	DRAFT. C009 activo; 19 u; 2,280.00	Agregar SKU003 × 2 con `add_item()`	Rechazo	Se lanza OrderItemLimitExceeded (19 + 2 = 21). Sigue con 19 u y un renglón.	Sin evento	Se suman unidades, no renglones; la lista no cambia.
E09 O010 test_e09	DRAFT. C010 no VIP; 2 u; exactamente 500.00	Evaluar descuento	0 %	0 %: paga 500.00.	Ninguno (consulta)	500 no es mayor que 500: el umbral es estricto.
E10 O011 test_e10	DRAFT. C011 VIP; 3 u; 1,500.00; sin promoción	Evaluar descuento	0 %	0 %: paga 1,500.00.	Ninguno (consulta)	Requiere al menos 4 unidades; tiene 3.
E11 O012 test_e11	DRAFT. C012 activo; 1 u; 850.00	`confirm()` y luego `ship("")`	Rechazo del envío	confirm() pasa a CONFIRMED; ship lanza InvalidOrderData: sin guía. Queda en CONFIRMED.	OrderConfirmed; el envío sin evento	La confirmación es válida; el envío exige guía.

Caso y prueba	Precondiciones	Operación	Resultado esperado	Resultado real	Evento	Explicación
E12 O013 test_e12	DRAFT. C001 activo; 1 u; 600.00	`confirm()` dos veces	Rechazo de la segunda	1.ª: CONFIRMED. 2.ª: InvalidOrderTransition (no se puede confirmar en estado CONFIRMED).	Un solo OrderConfirmed	Confirmar solo es válido desde DRAFT.
E13 O014 test_e13	DRAFT. C002 activo; 1 u; 600.00	`confirm()`, `ship()` y `cancel()`	Rechazo al cancelar	Llega a SHIPPED; cancel lanza InvalidOrderTransition. Queda en SHIPPED.	OrderConfirmed, OrderShipped; la cancelación sin evento	Cancelar solo procede desde CONFIRMED.
E14 O015 test_e14	DRAFT. C005 activo; 1 u; 450.00	`confirm()`, `ship()` y `return_order()`	Éxito	Termina en RETURNED.	OrderConfirmed, OrderShipped, OrderReturned	La devolución es válida desde SHIPPED.
E15 O016 test_e15	DRAFT. C007 activo; 1 u; 2,100.00	`confirm()`, `ship()` y `deliver()`	Éxito	Termina en DELIVERED.	OrderConfirmed, OrderShipped, OrderDelivered	La entrega es válida desde SHIPPED.

Tabla 7. *Bitácora de los 15 escenarios del paquete.*

9. API REST con FastAPI

La API (app/api.py) es solo la entrada a la lógica. Cada endpoint recibe la petición, llama al Application Service (app/services.py), que obtiene el pedido del repositorio en memoria e invoca el método del dominio, y devuelve un DTO de respuesta. Los endpoints no calculan descuentos ni cambian estados: si el dominio rechaza la operación, la excepción se traduce a un código HTTP en un solo lugar.

Método	Ruta	Descripción	Respuestas
GET	/orders	Lista los pedidos	200
GET	/orders/{order_id}	Detalle con artículos y eventos	200, 404
GET	/orders/{order_id}/discount	Descuento aplicable (no modifica el pedido)	200, 404
POST	/orders	Crea un pedido en DRAFT	201, 422
POST	/orders/{order_id}/items	Agrega un artículo del catálogo	200, 404, 409, 422
POST	/orders/{order_id}/confirm	Confirma el pedido	200, 404, 409, 422
POST	/orders/{order_id}/ship	Envía (body: tracking_number)	200, 404, 409, 422
POST	/orders/{order_id}/cancel	Cancela (body: reason)	200, 404, 409, 422
POST	/orders/{order_id}/deliver	Entrega	200, 404, 409
POST	/orders/{order_id}/return	Devuelve	200, 404, 409
POST	/fixtures/reload	Vuelve a cargar los CSV para probar casos separados	200

Tabla 8. *Endpoints de la API. Las operaciones que cambian el estado usan POST.*

app/api.py — controlador delgado

```
@app.exception_handler(OrderNotFound)           # 404: no existe
@app.exception_handler(InvalidOrderTransition)  # 409: transición inválida
@app.exception_handler(DomainException)         # 422: datos inválidos


@app.post("/orders/{order_id}/confirm", response_model=OrderDTO)
def confirm_order(order_id: str):
    return OrderDTO.from_order(service.confirm(order_id))
```

La documentación se genera automáticamente en /docs (Swagger) y el contrato en /openapi.json. En Swagger se probaron una operación exitosa y los tres rechazos que pide la práctica:

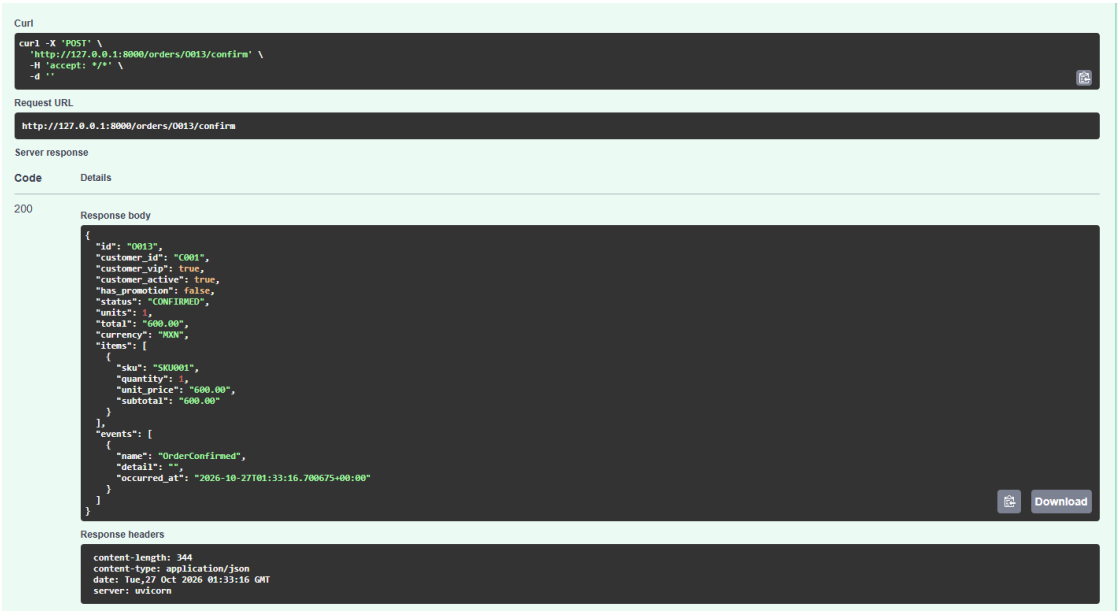


Figura 3. *Swagger: POST /orders/O013/confirm responde 200 y el pedido pasa a CONFIRMED con el evento OrderConfirmed.*

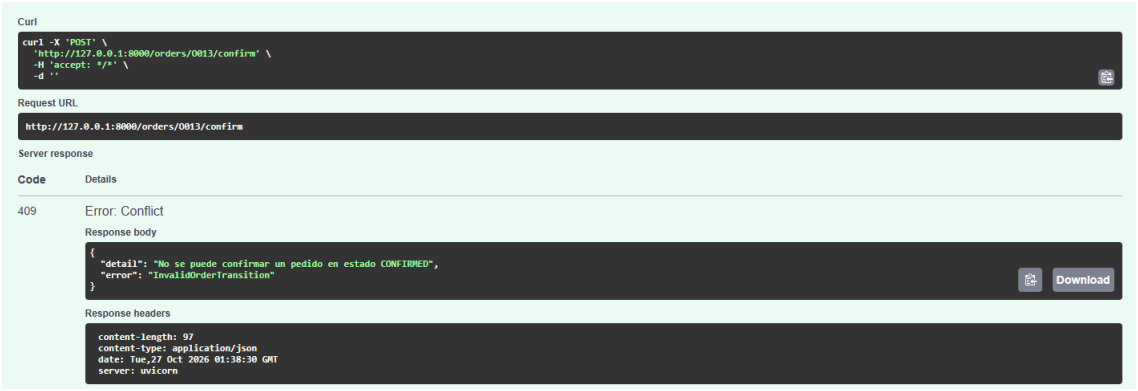


Figura 4. *Swagger: confirmar O013 por segunda vez responde 409 Conflict (InvalidOrderTransition).*

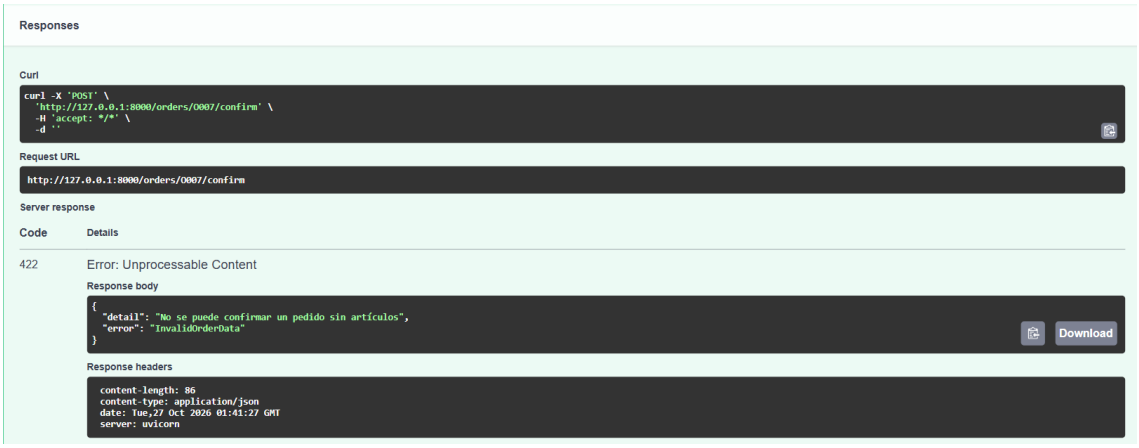


Figura 5. *Swagger: confirmar O007, pedido sin artículos, responde 422 (InvalidOrderData).*



Figura 6. *Swagger: confirmar O999, pedido inexistente, responde 404.*

10. Pruebas automáticas con pytest

Las pruebas siguen tres partes: preparar los datos, ejecutar la acción y comprobar el resultado. Cada prueba recibe una carga nueva de los CSV, y en las de la API los datos se reinician antes de cada caso. Además de comprobar que una regla funciona, comprueban que falla cuando corresponde y que, al fallar, el pedido no cambia de estado ni agrega eventos.

Archivo	Pruebas	Qué demuestra
tests/test_domain.py	35	Money (negativo, float, inmutabilidad, monedas), OrderItem, carga de los 16 pedidos, cálculo manual, límite de 20 unidades, cliente inactivo, pedido vacío, confirmación doble, envío sin guía, cancelación desde SHIPPED y estados finales.
tests/test_specifications.py	13	Cada Specification, la composición AND, OR y NOT, que la evaluación no modifica el pedido, los descuentos y la prioridad del 10 %.
tests/test_escenarios.py	15	Un caso por escenario, de E01 a E15.
tests/test_api.py	17	Respuestas 200, 201, 404, 409 y 422, documentación /docs y /openapi.json, flujo completo y recarga de datos.
Total	80	

Tabla 9. *Pruebas automáticas del proyecto.*

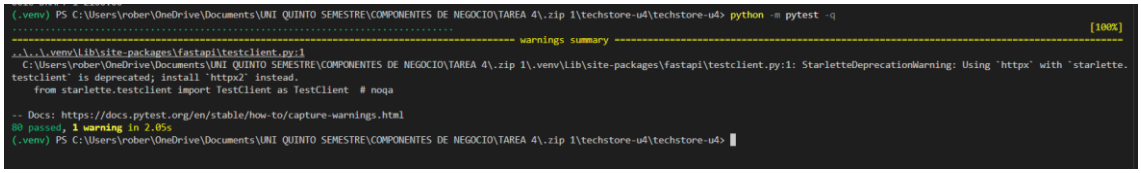


Figura 7. *Resultado de python -m pytest -q: 80 pruebas aprobadas.*

Las 80 pruebas pasan. La advertencia (warning) que aparece no es un fallo del proyecto: la emite la librería Starlette, en la que se basa FastAPI, para avisar que en versiones futuras su cliente de pruebas usará el paquete httpx2 en lugar de httpx. Se conservó httpx porque es el paquete que indica la práctica, y no afecta el resultado.

11. Lista de verificación

Punto de la práctica	Cumple	Evidencia
python cargar_csv.py se ejecuta sin errores	Sí	Figura 1
O001 muestra 4 unidades y 2,100.00 MXN	Sí	Figura 1, Tabla 2, test_pedido_o001_inicia_draft
Regla del 10 %: VIP, > 500, ≥ 4 unidades y sin promoción	Sí	TEN_PERCENT_RULE, E01, E09, E10
Regla del 5 %: > 5000 y sin promoción	Sí	FIVE_PERCENT_RULE, E05
Una operación inválida no cambia el estado ni añade eventos	Sí	E03, E06, E08, E11, E12, E13
La API responde 404, 409 y 422 en los casos indicados	Sí	Figuras 4 a 6, test_api.py
pytest termina sin fallos	Sí	Figura 7
El repositorio contiene README y evidencias	Sí	Repositorio de GitHub

Tabla 10. *Lista de verificación de la sección 14 de la práctica.*

12. Preguntas para el reporte

¿Por qué se usa Decimal en lugar de float para precios?

Porque float guarda los números en binario y muchos decimales, como 0.1, no tienen representación exacta; al sumar y multiplicar se acumulan errores de redondeo (0.1 + 0.2 da 0.30000000000000004). Decimal representa exactamente los centavos y permite redondear con una regla definida, algo indispensable cuando se calculan totales y descuentos. Por eso Money rechaza float, y los precios se construyen con Decimal("600.00") a partir del texto del CSV, no desde un float.

¿Cuál es la diferencia entre contar renglones y sumar unidades?

Contar renglones es contar cuántos productos distintos tiene el pedido (len(items)); sumar unidades es sumar la cantidad de cada renglón. El límite del negocio es de 20 unidades: O008 tiene un solo renglón con 20 unidades y es válido, mientras que agregar 2 unidades a O009, que ya tiene 19, se rechaza aunque solo serían dos renglones. Lo mismo aplica a la regla de «al menos 4 unidades»: O011 tiene 2 renglones pero 3 unidades.

¿Qué ventaja tiene encapsular una regla en Specification?

Cada condición queda con un nombre del negocio, se prueba por separado y se reutiliza: MinimumAmountSpec y ~ActivePromotionSpec se usan tanto en la regla del 10 % como

en la del 5 %. Las reglas se combinan con AND, OR y NOT sin if anidados, y si cambia un umbral se modifica en un solo lugar. Además, la Specification solo evalúa y no modifica el pedido, por lo que puede usarse para validar, filtrar consultas o calcular descuentos.

¿Por qué los eventos se registran solo después de una transición válida?

Porque un Domain Event anuncia un hecho que ya ocurrió. Si se registrara antes de validar y la operación fallara, otros procesos reaccionarían a algo que nunca pasó; por ejemplo, se enviaría un correo de «pedido confirmado» para un pedido vacío. En E12 la segunda confirmación falla y la lista sigue con un solo OrderConfirmed.

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Todo excepto app/api.py: el dominio (domain.py), las Specifications (specifications.py) y el Application Service con su repositorio (services.py) no importan nada de FastAPI. Una aplicación de consola, un proceso por lotes o una interfaz gráfica podrían llamar a los mismos métodos; de hecho, ejecutar_escenarios.py y las pruebas del dominio ya usan esa lógica sin servidor web. Solo habría que escribir el nuevo adaptador de entrada.

13. Reflexión: errores encontrados

Error encontrado	Cómo se solucionó	Qué aprendimos
Al ejecutar python --version, Windows respondió «No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store». Python no estaba instalado; el comando era solo un acceso directo de Windows.	Se instaló Python 3.13.17 desde python.org marcando «Add python.exe to PATH» y se reinició VS Code. No se eligió la versión más reciente que ofrecía la página, sino la 3.13 que usó el docente.	Antes de programar hay que verificar el ambiente: versión del lenguaje, PATH y entorno virtual. Una versión recién publicada puede no ser compatible con todas las librerías.
Al activar el entorno virtual con .venv\Scripts\Activate.ps1, PowerShell respondió «No se puede cargar el archivo … Activate.ps1 porque la ejecución de scripts está deshabilitada en este sistema».	Se ejecutó Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, que permite ejecutar scripts locales solo para el usuario actual, y se volvió a activar el entorno; apareció (.venv) al inicio de la línea.	Windows bloquea los scripts de PowerShell por seguridad. Hay que entender el mensaje antes de cambiar una configuración y limitar el cambio al usuario; otra opción era activar el entorno desde CMD con activate.bat.

Error encontrado	Cómo se solucionó	Qué aprendimos
El escenario E09 (O010, total exactamente 500) daba 0 %, pero el cliente C010 no es VIP: el descuento sería 0 % aunque el umbral estuviera mal programado con >=. El escenario por sí solo no demostraba la regla.	Se agregó una prueba directa de MinimumAmountSpec(500): con O010 (500.00) debe ser falsa y con 500.01 verdadera.	Que una prueba pase no significa que pruebe lo que se quiere; hay que revisar qué condición aísla cada caso, sobre todo en los valores frontera.

Tabla 11. *Tres errores encontrados durante la actividad.*

La siguiente figura muestra el segundo error tal como apareció en la terminal de VS Code:

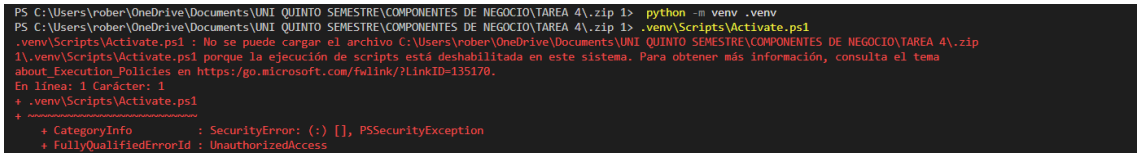


Figura 8. *Error al activar el entorno virtual: la ejecución de scripts estaba deshabilitada en PowerShell.*

Uso de inteligencia artificial. Como propone el manual (tema 8, Vibe Coding), se usó IA para generar un primer borrador del código, las pruebas y el diagrama a partir de una especificación precisa de estados, transiciones, precondiciones y eventos. Cada parte se revisó contra las reglas del paquete: se verificó qué método protege cada regla, se ejecutaron las pruebas y los 15 escenarios, y se corrigió lo que no coincidía, como la etiqueta del estado inicial que se encimaba en el primer diagrama, el total «0» del pedido vacío que se cambió a «0.00» o la prueba del umbral de E09 de la tabla anterior.

Conclusiones:

Esta actividad convirtió el diseño de la unidad anterior en código que se puede ejecutar y comprobar. Las reglas de TechStore quedaron en el dominio: Money impide importes negativos y el uso de float, OrderItem calcula su subtotal, y Order controla el límite de 20 unidades y su propio ciclo de vida. Como la máquina de estados vive en el pedido, ninguna capa puede saltarse una transición; la API, las pruebas y el script de escenarios obtienen exactamente el mismo comportamiento.

El Specification Pattern permitió expresar los descuentos con el mismo lenguaje del negocio y combinarlos con AND, OR y NOT, y los Domain Events dejaron registro de cada hecho ocurrido solo cuando la operación fue válida. La API quedó como una capa delgada que traduce las excepciones del dominio a 404, 409 y 422. Los 15 escenarios dieron el resultado esperado sin modificar los datos ni las reglas, y las 80 pruebas automáticas respaldan cada regla, tanto en los casos válidos como en los rechazos.

Conclusiones individuales:

Roberto Carlos Sánchez Flores

En esta actividad entendí la diferencia entre que un programa funcione y que proteja las reglas del negocio. Al principio veía el límite de 20 unidades o la confirmación como simples validaciones, pero al ponerlas dentro de Order me di cuenta de que nadie puede saltárselas, ni desde la API ni desde una prueba. También me quedó claro por qué el dinero se maneja con Decimal: el caso de O010, con un total de exactamente 500, me hizo ver lo importante que es cuidar los valores frontera y no solo los casos normales.

Donovan Adrian Serrato Lores

Lo que más me sirvió fue el Specification Pattern. Antes habría escrito la regla del descuento con varios if anidados en el endpoint; ahora entiendo que cada condición puede ser un objeto con nombre, probarse sola y combinarse con AND, OR y NOT. También aprendí a mantener delgada la API: el controlador solo llama al servicio y las excepciones del dominio se convierten en 404, 409 o 422. Ver en Swagger el 409 al confirmar dos veces me ayudó a entender que la regla está en el dominio y no en la pantalla.

Laura Sofía Vargas Ramírez

Para mí lo más importante fue la máquina de estados y los Domain Events. Dibujar primero el diagrama me ayudó a ver qué transiciones existen y cuáles no, como cancelar un pedido que ya se envió. Aprendí que el evento se registra al final, solo cuando la operación fue válida, porque anuncia algo que ya ocurrió. También aprendí a revisar con cuidado lo que genera la IA y lo que muestran las pruebas: un escenario puede pasar sin demostrar la regla que parece probar, y por eso agregamos pruebas que aíslan cada condición.

Repositorio de GitHub:

El repositorio contiene las carpetas app, tests y datos, el cargador del paquete, el script de escenarios, el diagrama de estados en docs y un README con los pasos de instalación, ejecución, reglas y endpoints.

https://github.com/RobertocSanchezf23/techstore_u4_equipo-12